Introduction to Neural Networks

Avinash Sharma

CSE Department, IIT Jodhpur

Image courtesy: https://www.etsy.com/in-en/listing/1607436850/third-eye-activation-spiritual-awakening

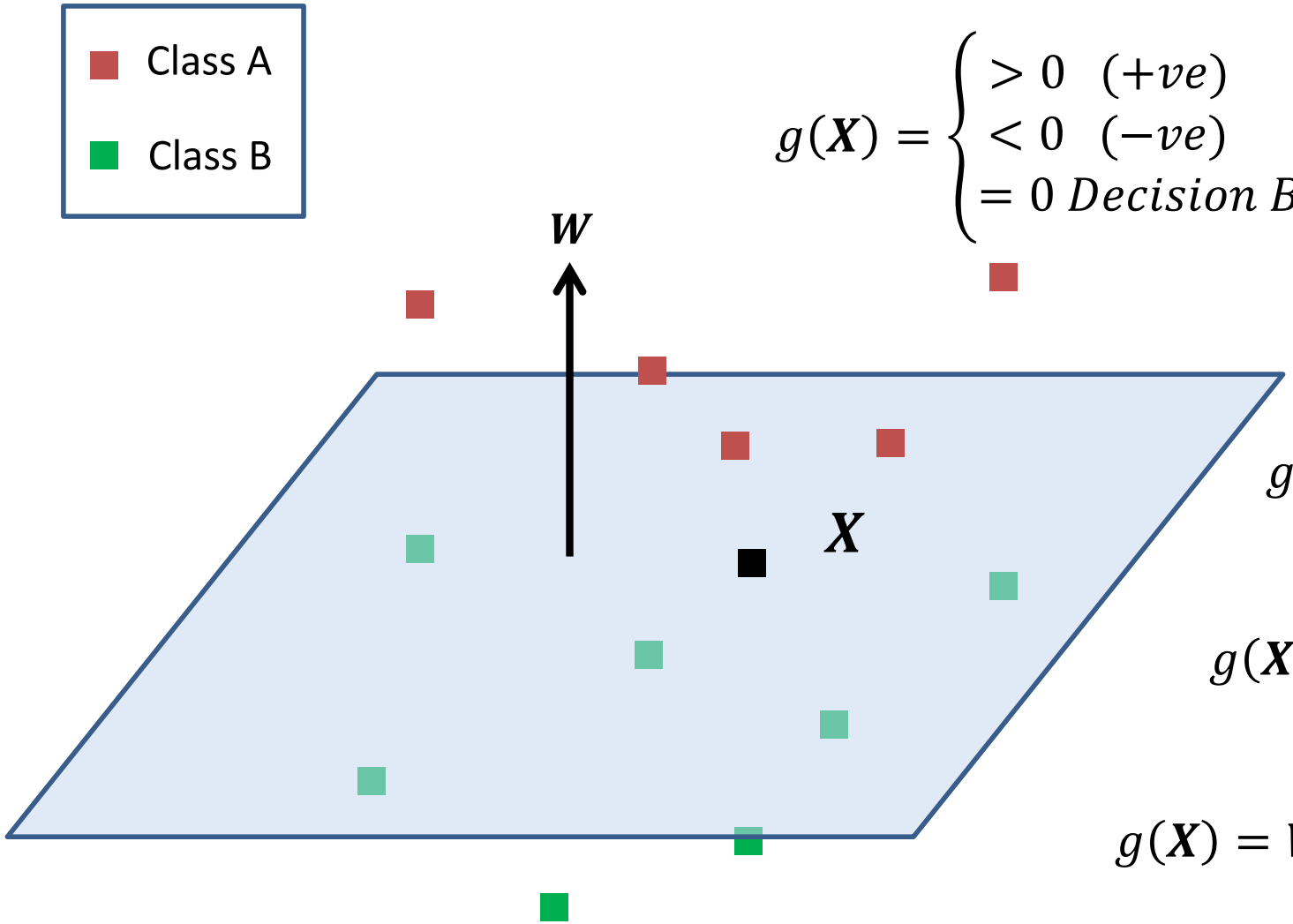
Lecture Outline

- Intro to Perceptron
- Multi-Layer Perceptron (MLP)
- Backpropagation for learning MLP weights

Introduction to Perceptron

- The perceptron is the basic building block of Neural Networks.
- Perceptron is essentially a Linear Discriminant Function.

Introduction Perceptron



$$g(\boldsymbol{X}) = \begin{cases} > 0 \quad (+ve) & class\ A \\ < 0 \quad (-ve) & class\ B \\ = 0\ Decision\ Boundary \end{cases}$$

$$g(\boldsymbol{X}) = 0$$

$$g(\boldsymbol{X}) = \boldsymbol{W}^T\boldsymbol{X}$$

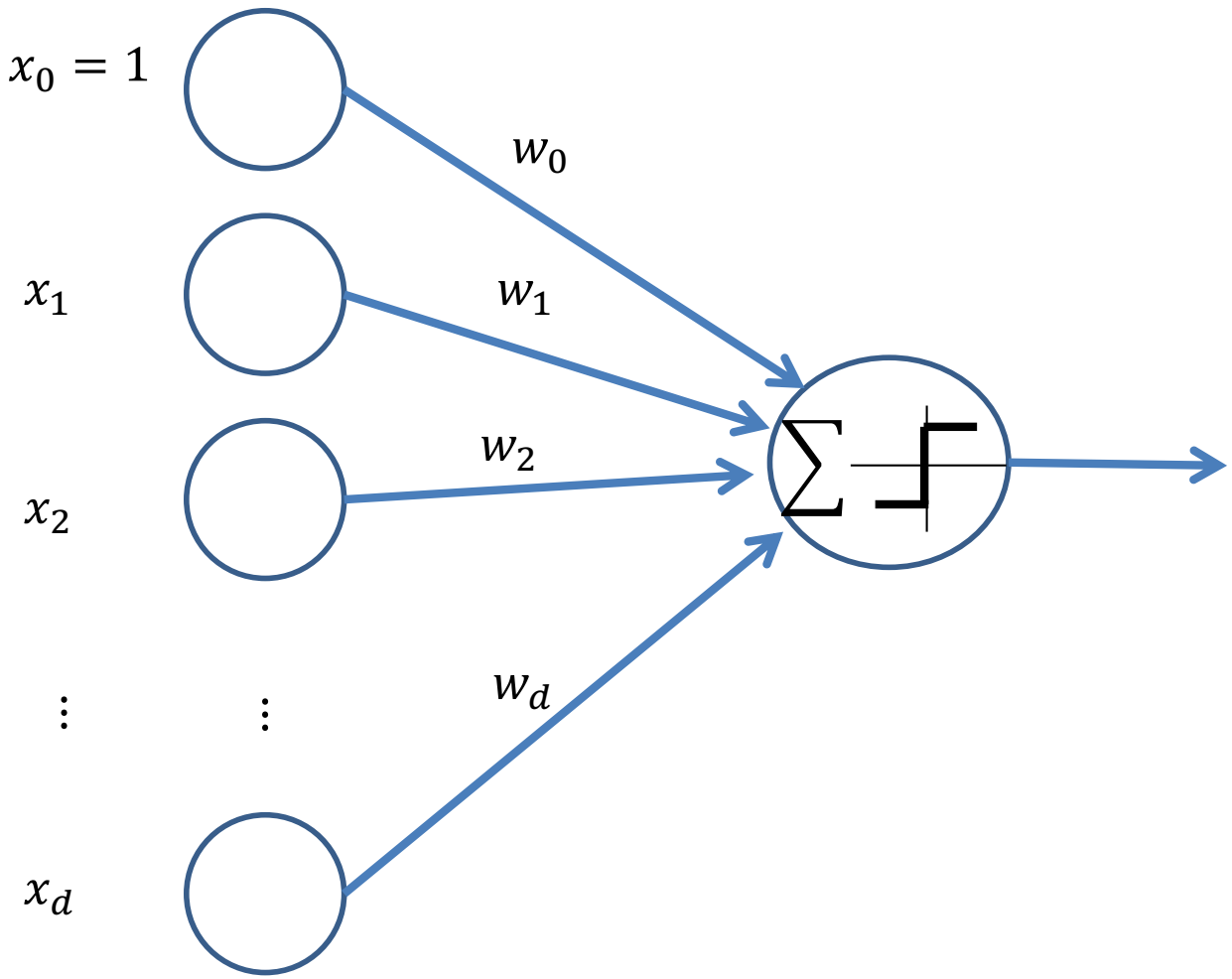
$$g(\boldsymbol{X}) = \boldsymbol{W}^T\boldsymbol{X} + w_0$$

The Perceptron

$$g(\boldsymbol{X}) = \boldsymbol{W}^T \boldsymbol{X} + w_0$$

$$\boldsymbol{X} = \begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix}$$

$$\boldsymbol{W} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix}$$



$$f(g(\boldsymbol{X}))$$

Introduction to Perceptron

- The perceptron is the basic building block of Neural Networks.
- Perceptron is essentially a Linear Discriminant Function (LDF).
- Thus, the normal vector of the linear hyperplane (i.e., $\boldsymbol{W}$) and bias term (i.e., w_0) are the learnable parameters.
- Even though it has a non-linearity (e.g., Signum), ***the perceptron is a linear classifier.***

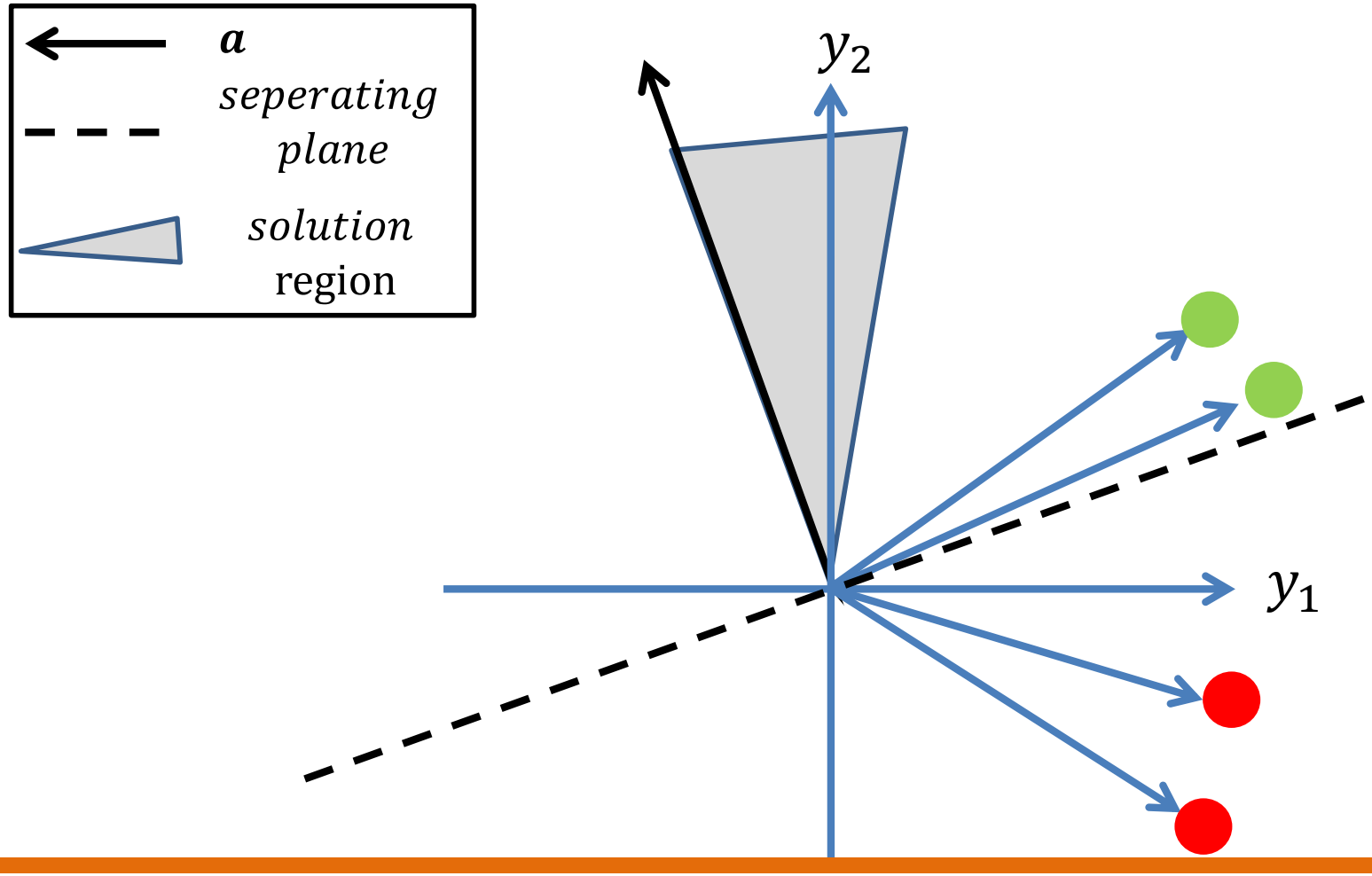
Introduction to Perceptron

- Decision boundary surface (hyperplane) $g(\boldsymbol{X}) = 0$ divides feature space into two regions.
- Orientation of the boundary surface is decided by the normal vector $\boldsymbol{w}$.
- Location of the boundary surface is determined by the bias term w_0 .
- $g(\boldsymbol{X})$ is proportional to distance of $\boldsymbol{X}$ from the boundary surface.
- $g(\boldsymbol{X}) > 0$ positive side and $g(\boldsymbol{X}) < 0$ negative side.

Learning the Perceptron Weights

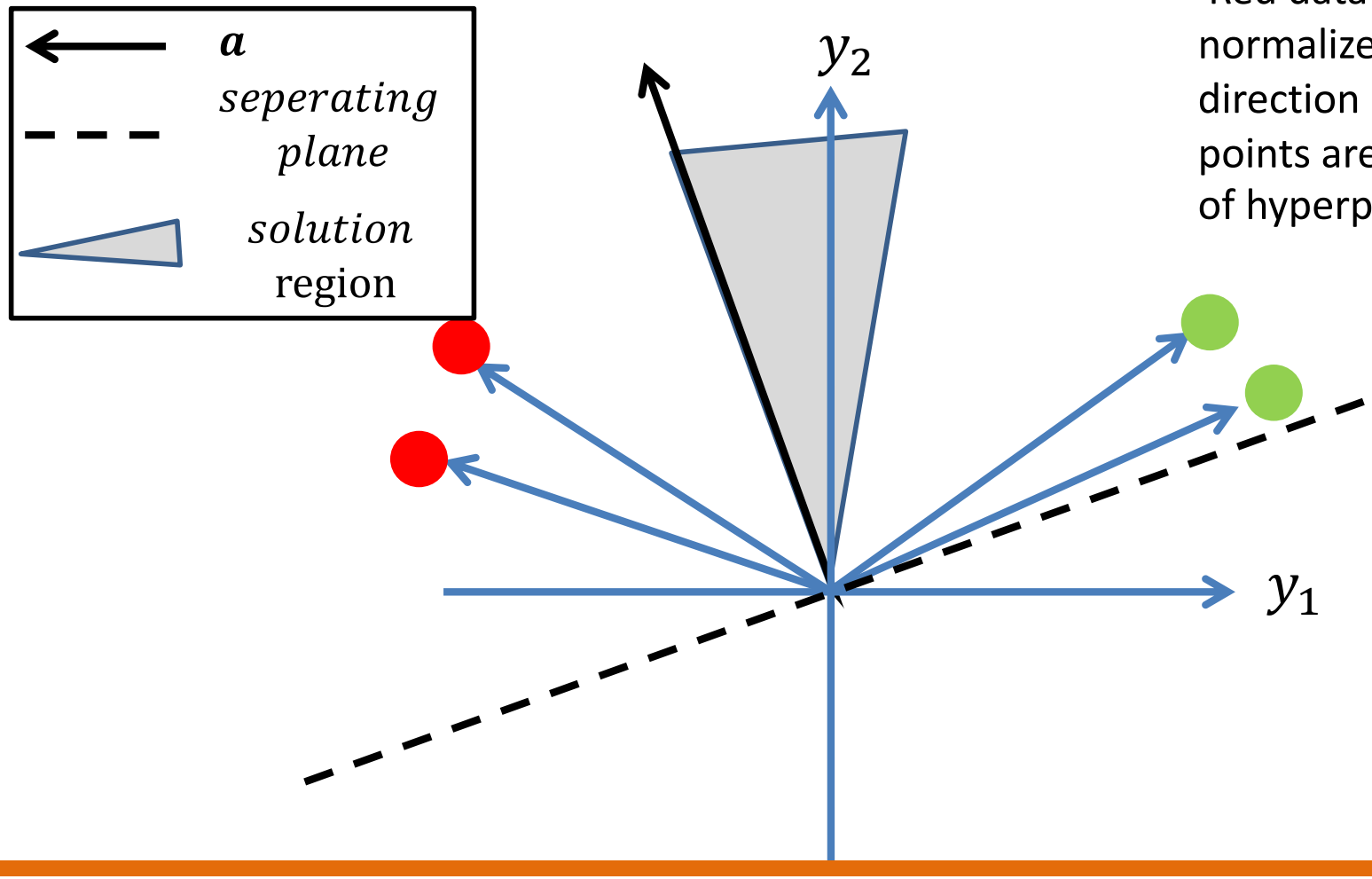
Two-Category Linearly Separable Case

$$g(\boldsymbol{X}) = \boldsymbol{a}^T \boldsymbol{Y} = \sum_{i=1}^{\hat{d}} a_i y_i \quad = \begin{cases} > 0 \quad (+ve) & class\ A \\ < 0 \quad (-ve) & class\ B \\ = 0\ Decision\ Boundary \end{cases}$$



Two-Category Linearly Separable Case

$$g(\boldsymbol{X}) = \boldsymbol{a}^T \boldsymbol{Y} = \sum_{i=1}^{\hat{d}} a_i y_i > 0$$



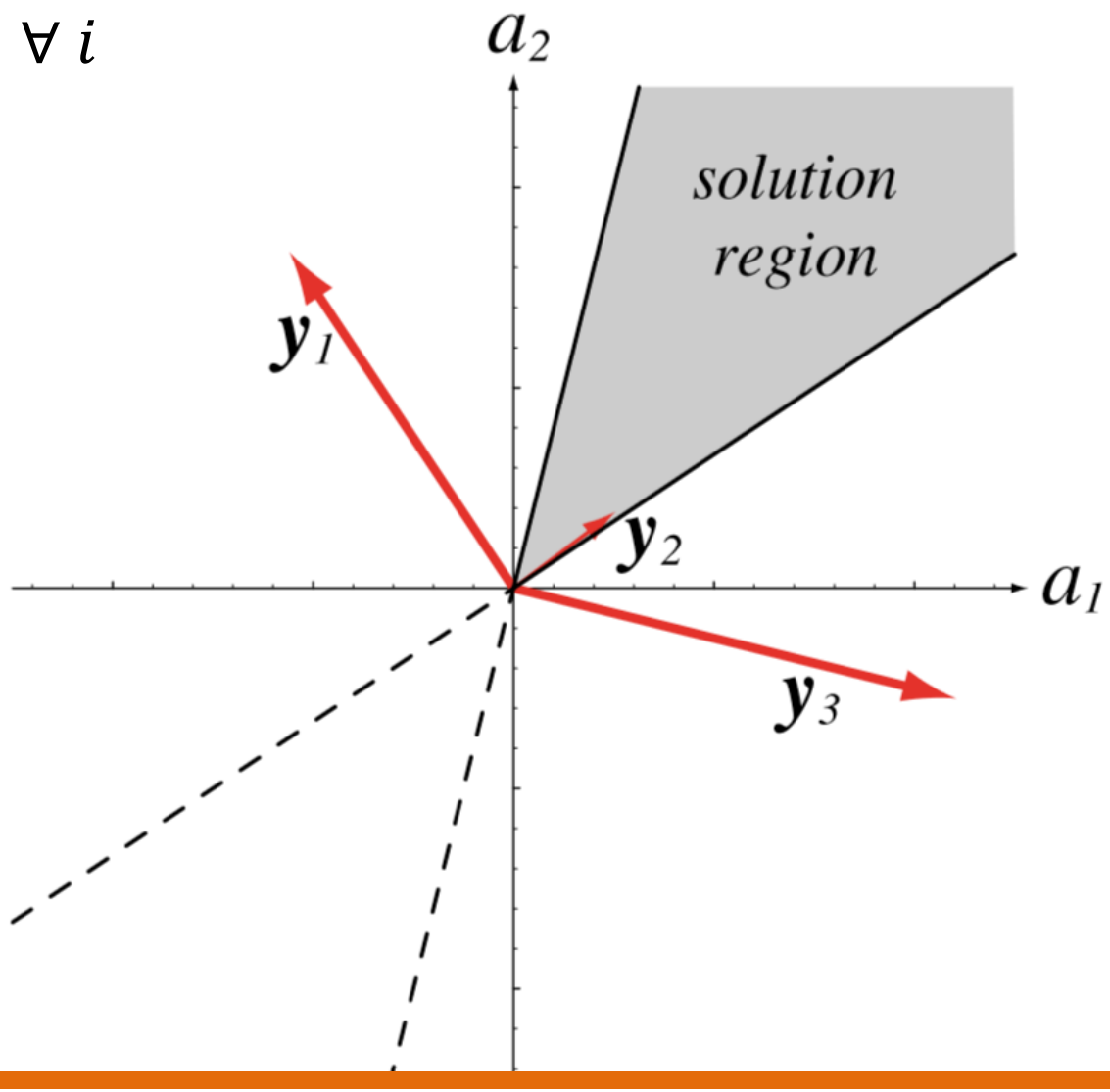
Normalized Case

- Red data points are normalized by flipping their direction so that all data points are on the one side of hyperplane.

Two-Category Linearly Separable Case

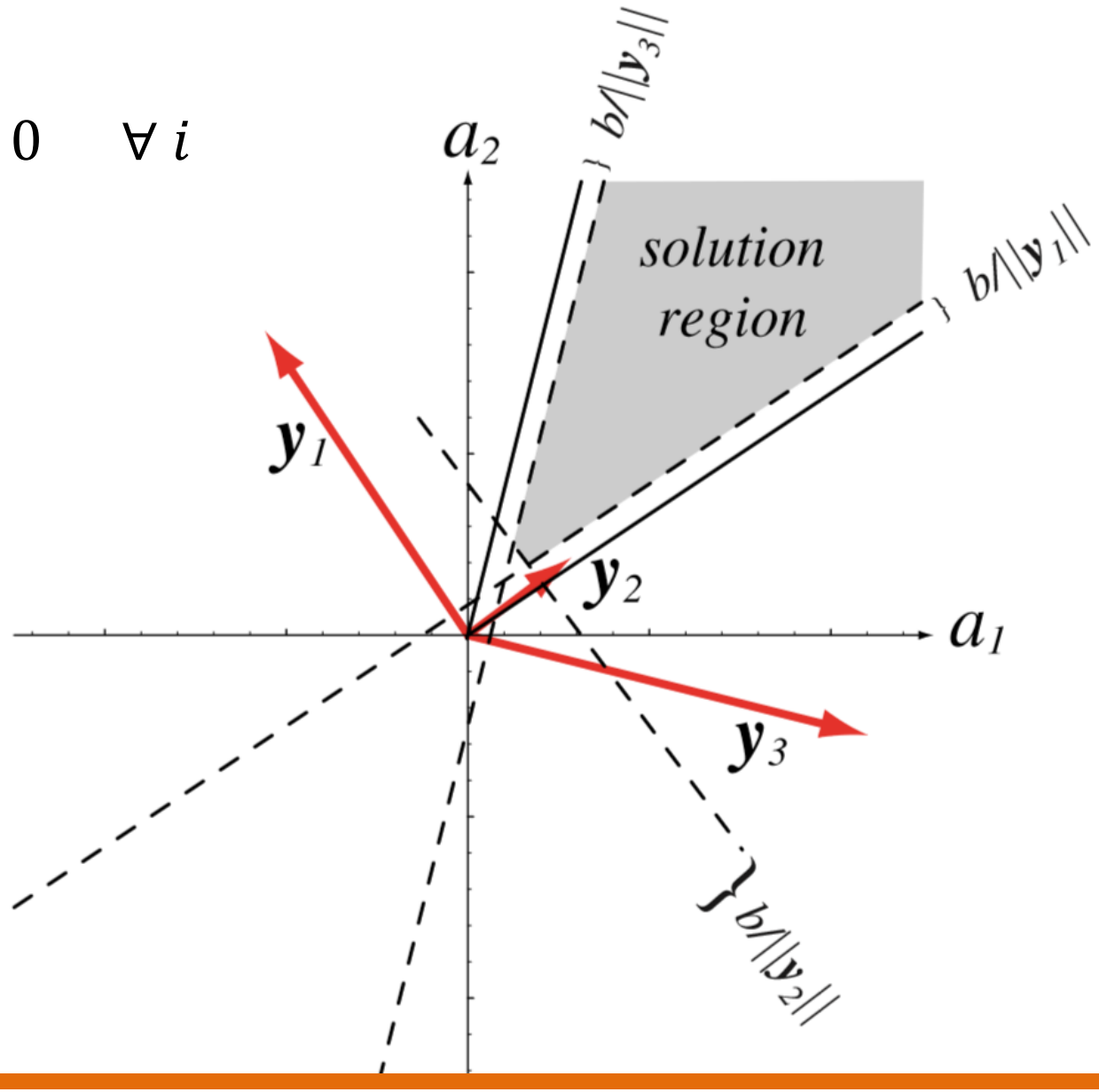
$$\boldsymbol{a}^T \boldsymbol{y}_i > 0 \qquad \forall\, i$$

Data vector



Two-Category Linearly Separable Case

$$\boldsymbol{a}^T \boldsymbol{y}_i \geq b > 0 \quad \forall\, i$$



Learning LDF: Basic Gradient Descend

- Define a scalar function $J(\boldsymbol{a})$ which captures classification error for specific boundary plane described by parameter $\boldsymbol{a}$

- Minimize $J(\boldsymbol{a})$ using ***gradient descent***.
  - Start with arbitrary value of $\boldsymbol{a}(1)$ for $k = 1$.
  - Iteratively refine estimate of $\boldsymbol{a}$:

$$\boldsymbol{a}(k+1) = \boldsymbol{a}(k) - \eta(k)\nabla J(\boldsymbol{a}(k))$$

- η is the positive scale factor known as ***learning rate***
  - A too small η makes the convergence very slow
  - A too large η can diverge due to overshooting correct.

Basic Gradient Descend Algorithm

1. Initialize $\boldsymbol{a}, \theta$ (threshold), $\eta(.), k = 0$

2. do $k = k + 1$

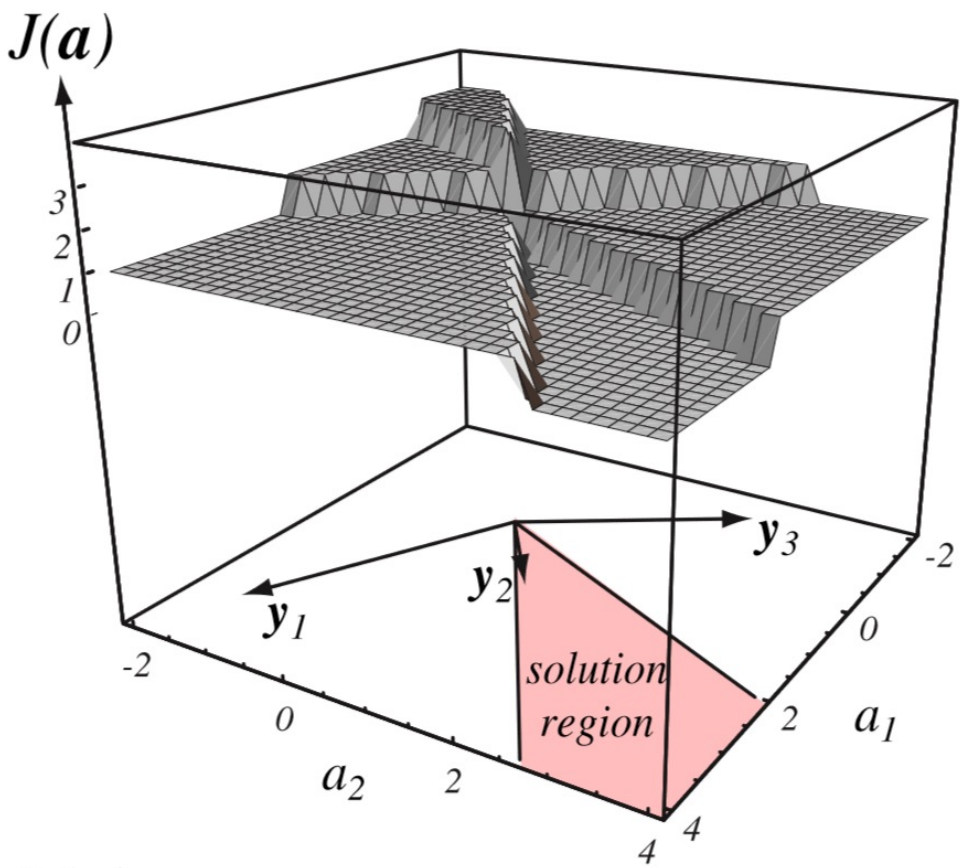
$$\boldsymbol{a}(k+1) = \boldsymbol{a}(k) - \eta(k)\nabla J(\boldsymbol{a}(k))$$

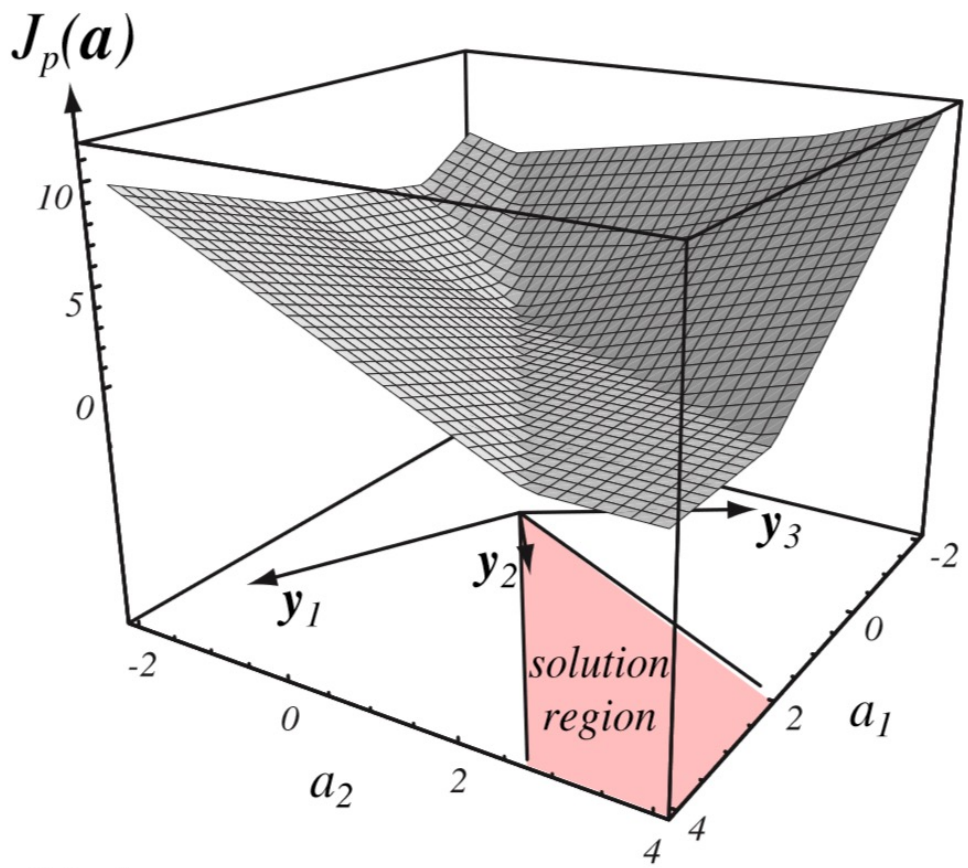
3. untill $|\eta(k)\nabla J(\boldsymbol{a}(k))| < \theta$

4. return $\boldsymbol{a}$

Perceptron Criterion Function

- Discrete v/s continuous function





Perceptron Criterion Function

- Discrete v/s continuous function
- $J_p(\boldsymbol{a}) = \sum_{\boldsymbol{y} \in \mathcal{Y}} (-\boldsymbol{a}^T \boldsymbol{y})$ where $\mathcal{Y}$ **<u>is the set of misclassified samples</u>**
- $J_p(\boldsymbol{a})$ is proportional to to the sum of the distances from all the miss classified samples to the decision boundary.
- Derivative of $J_p(\boldsymbol{a})$

$$\nabla J_p(\boldsymbol{a}) = \sum_{\boldsymbol{y} \in \mathcal{Y}} (-\boldsymbol{y})$$

- $\boldsymbol{a}(k+1) = \boldsymbol{a}(k) + \eta(k) \sum_{\boldsymbol{y} \in \mathcal{Y}_k} \boldsymbol{y}$

Batch Perceptron

1. Initialize $\boldsymbol{a}, \theta$ (threshold), $\eta(.), k = 0$

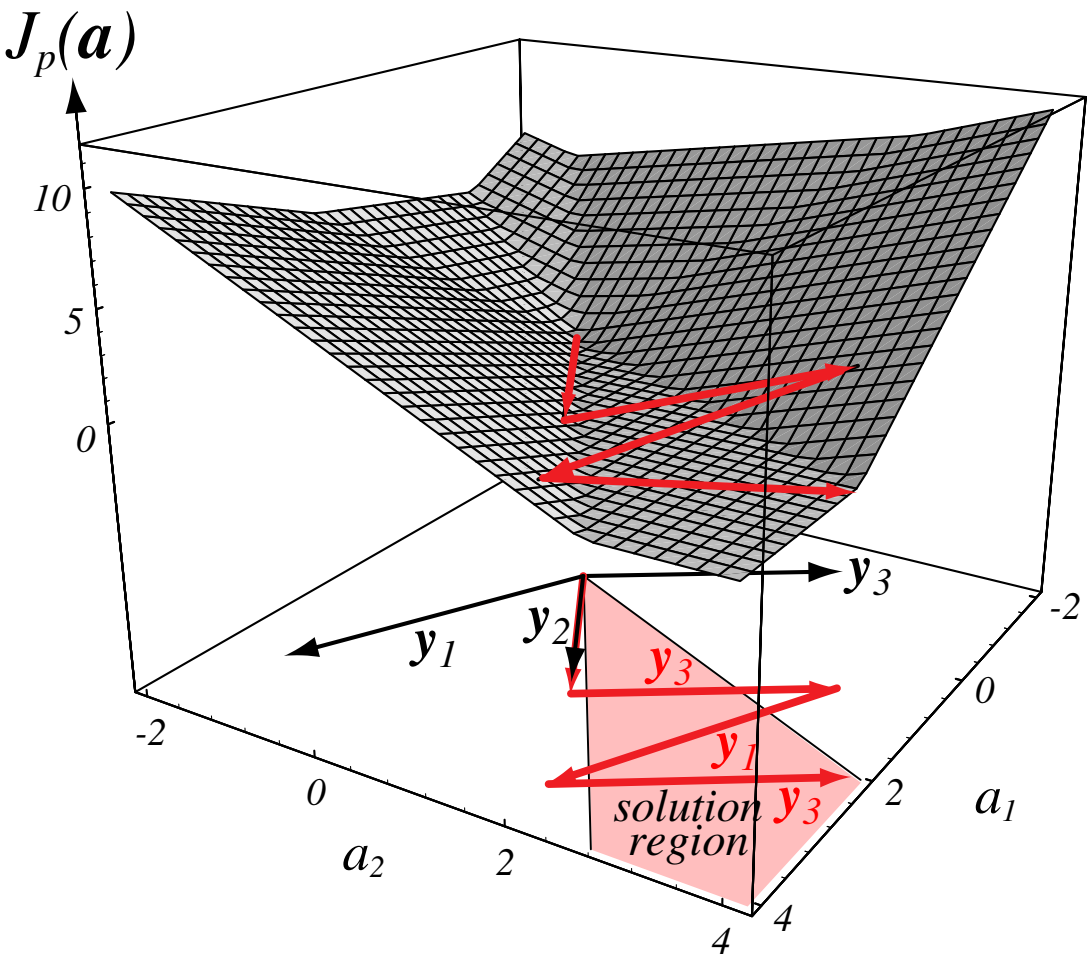
2. do $k = k + 1$

$$\boldsymbol{a}(k+1) = \boldsymbol{a}(k) + \eta(k) \sum_{\boldsymbol{y} \in \mathcal{Y}_k} \boldsymbol{y}$$

3. untill $\left|\eta(k) \sum_{\boldsymbol{y} \in \mathcal{Y}_k} (-\boldsymbol{y})\right| < \theta$

4. return $\boldsymbol{a}$

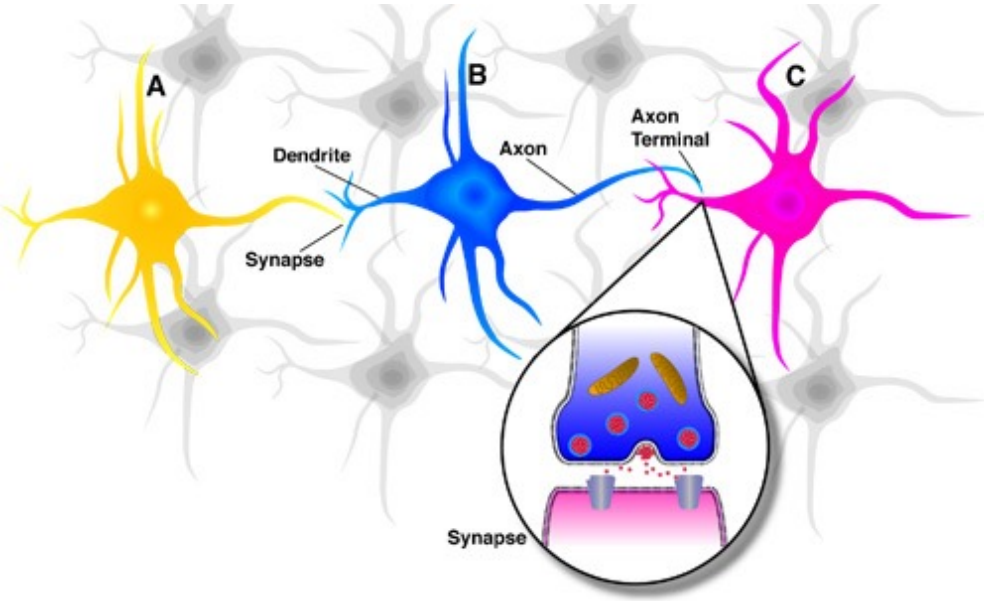
Visualizing the Learning



Introduction to Neural Networks

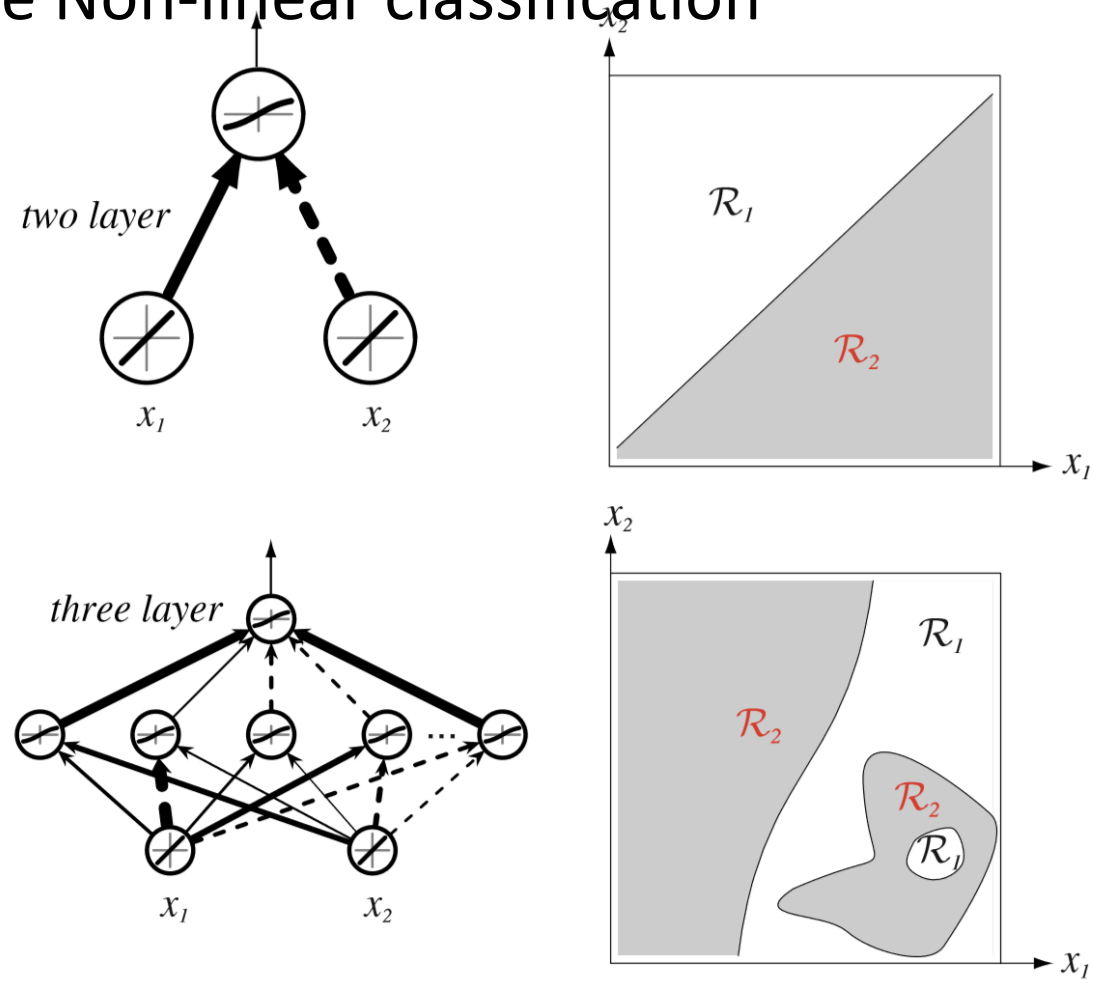
Motivation from Biology

- Brain is the key component in our body as it enables learning.
- It has a collection of about 10 billion interconnected **neurons**.
- A neuron receives input from other neurons (generally thousands) from its **synapses**
- Inputs are approximately summed
- Once the this sum exceeds a threshold the neuron sends an electrical spike that travels through the axon to the next neuron(s)



Introduction to Neural Networks

- Natural Extension of LDF (Perceptron) to Multi Layer Perceptron for the Non-linear classification



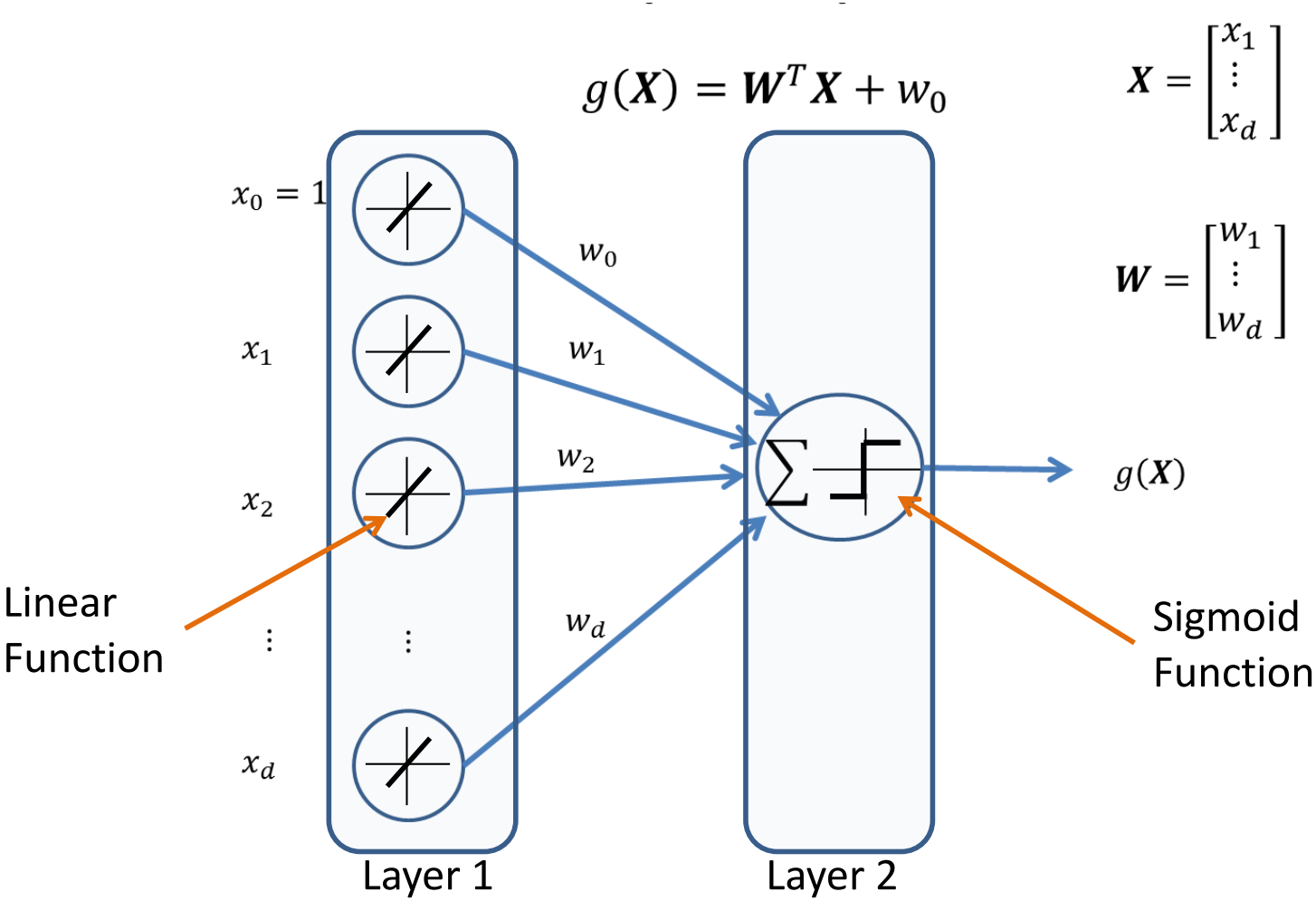
Introduction to Neural Networks

- Natural Extension of LDF (Perceptron) for Non-linear classification
- Temporal History

1960	• Hand Coded NN (No Learning)
1985	• Back Propagation NN (Chain Rule of GD)
1990	• Kernel Methods (Improved training)
2000	• Convolutional NN
2010	• Deep Learning (Scalable Learning)

Construction of Two Layer Perceptron

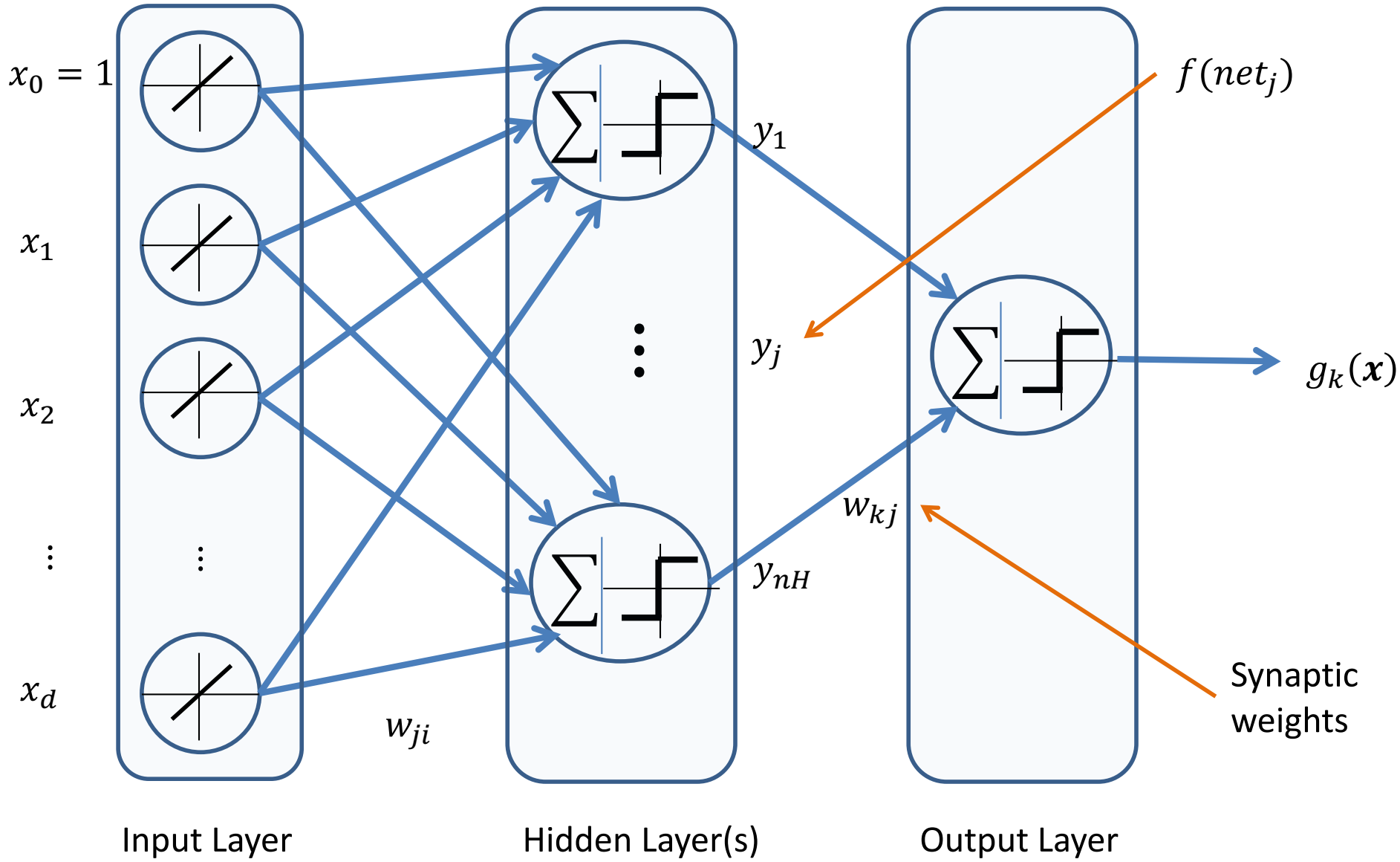
- Construction of a Two Layer Neural Network



Construction of Multi Layer Perceptron

- Hidden Layer
- Bias Unit
- Neuron
- Net Activation
- Synapses/ Synaptic Weights
- Activation Function

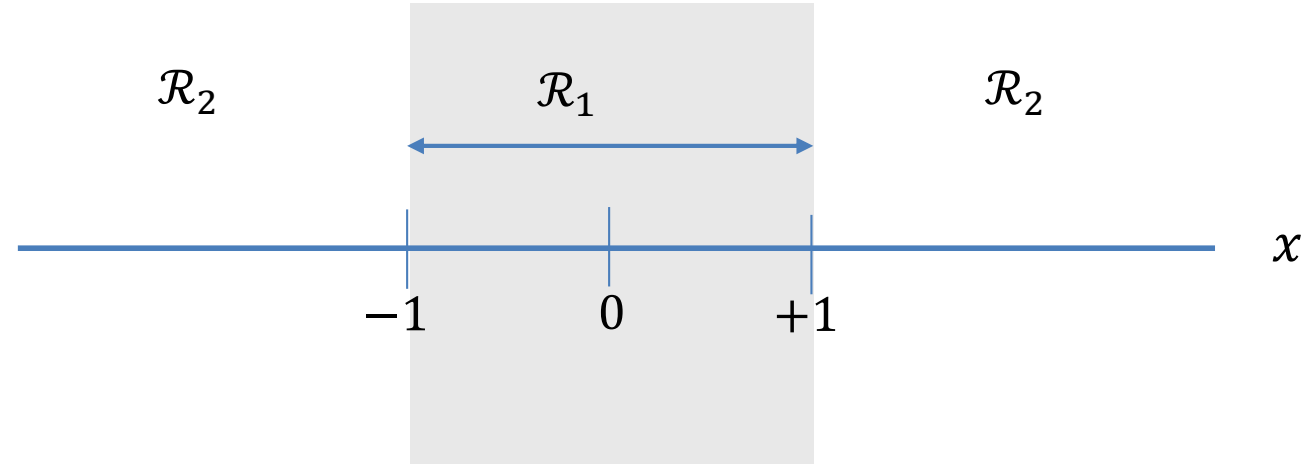
Construction of MLP classifier

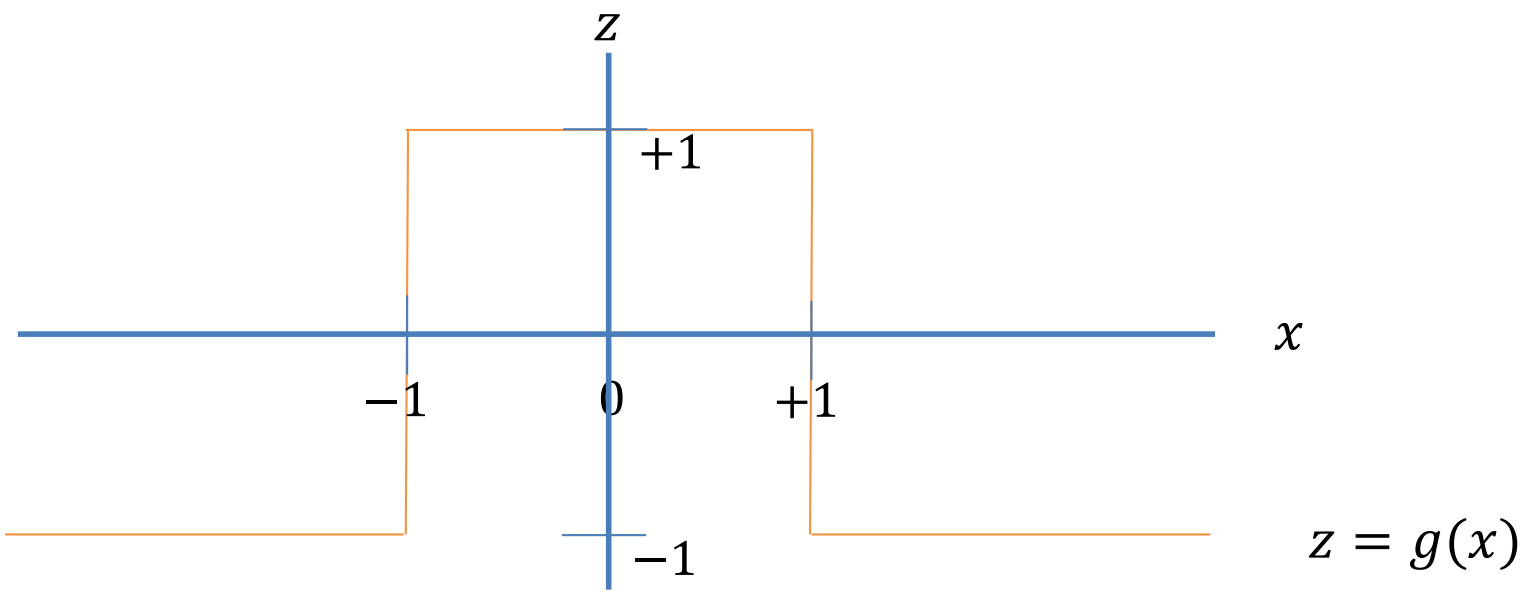


Feed Forward Operation in MLP

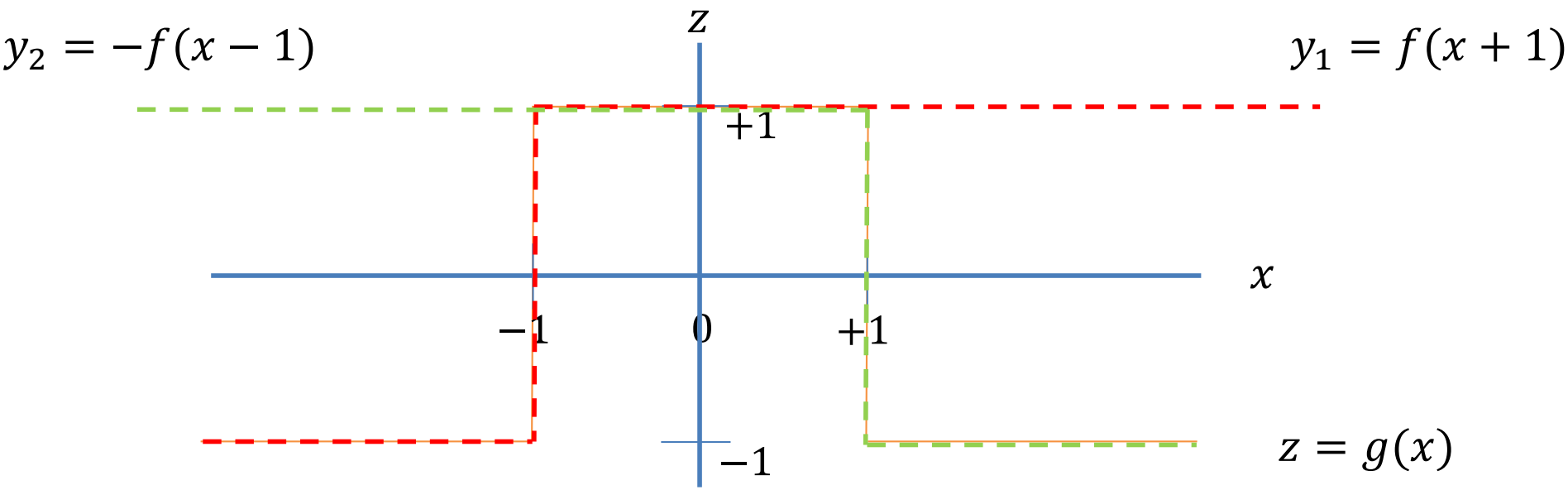
- $net_j = \sum_{i=1}^{d} x_i w_{ji} + w_{j0} = \sum_{i=0}^{d} x_i w_{ji} = \mathbf{w}^T{}_j \ \mathbf{x}$
- $y_j = f(net_j)$
- $y_j = sgn(net_j)$
- $net_k = \sum_{j=1}^{nH} y_j w_{kj} + w_{k0} = \sum_{j=0}^{nH} y_j w_{kj} = \mathbf{w}^T{}_k \ \boldsymbol{y}$
- $z_k = f(net_k) = sgn(net_k)$
- $g_k(\mathbf{x}) = z_k = \sum_{j=1}^{nH} w_{kj} f(\sum_{i=1}^{d} x_i w_{ji} + w_{j0}) + w_{k0}$

Modelling the Non-linearity with MLP





Modelling the Non-linearity with MLP



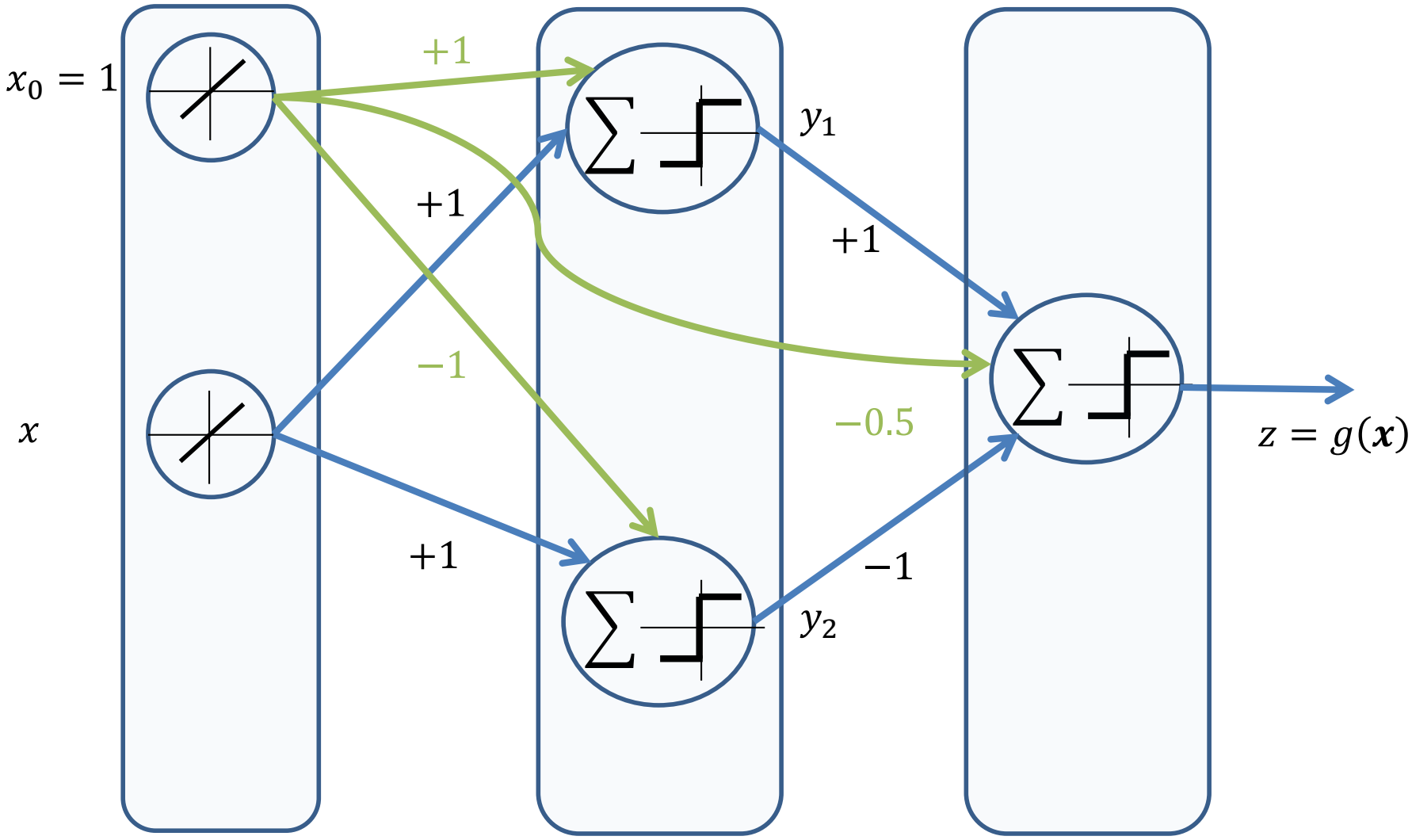
$y_1 = sgn(w * x + b)$ $\quad$ $w = 1, b = +1$

$y_2 = sgn(w * x + b)$ $\quad$ $w = 1, b = -1$

$z = g(x) = sgn(w_1 y_1 + w_2 y_2 + b)$ $\quad$ $w_1 = +1, w_2 = -1, b = -0.5$

Modelling the Non-linearity with MLP

x
+2
+1.2
+0.5
0
-0.5
-1.2
-2

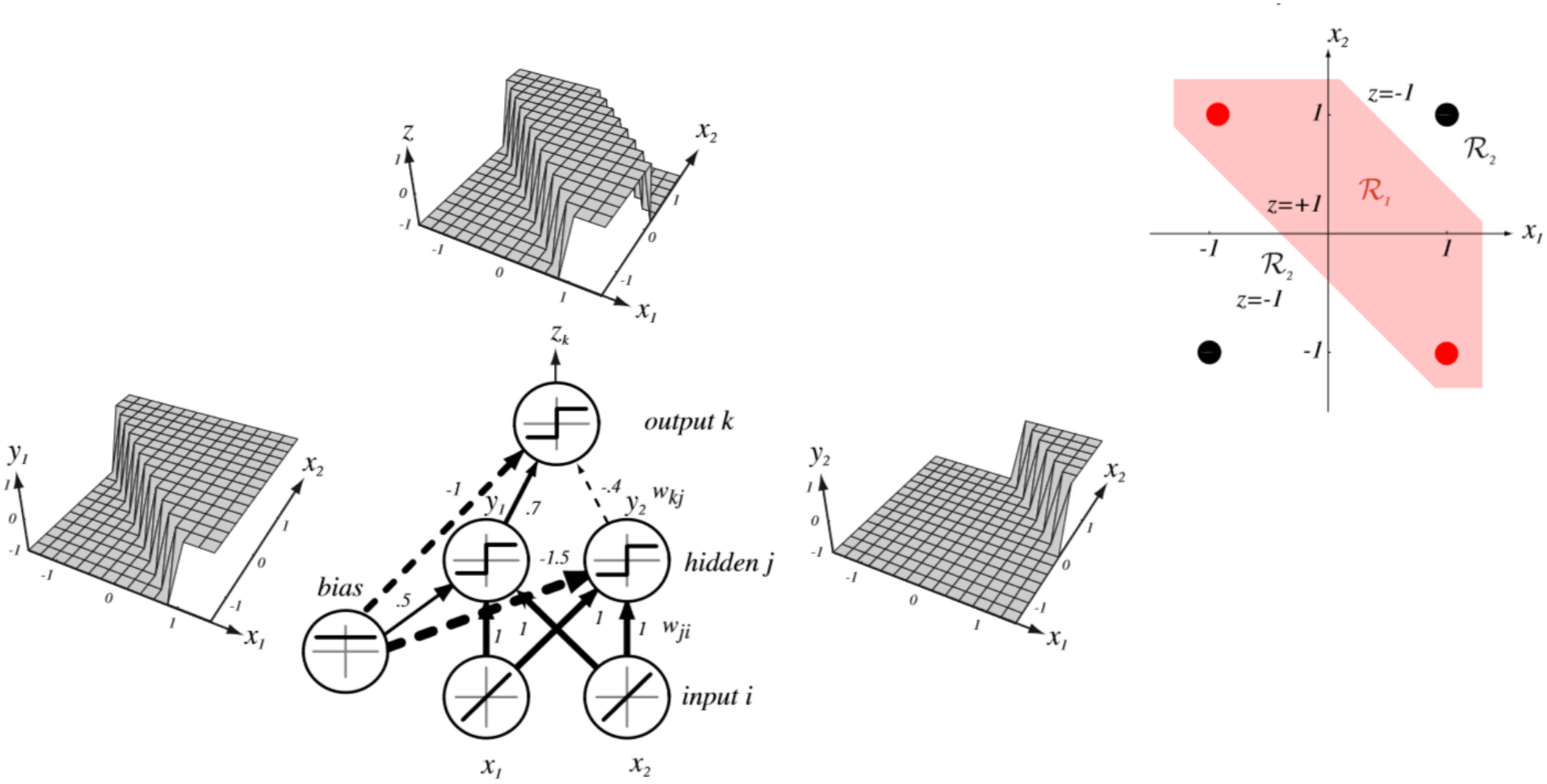


Input Layer Hidden Layer(s) Output Layer

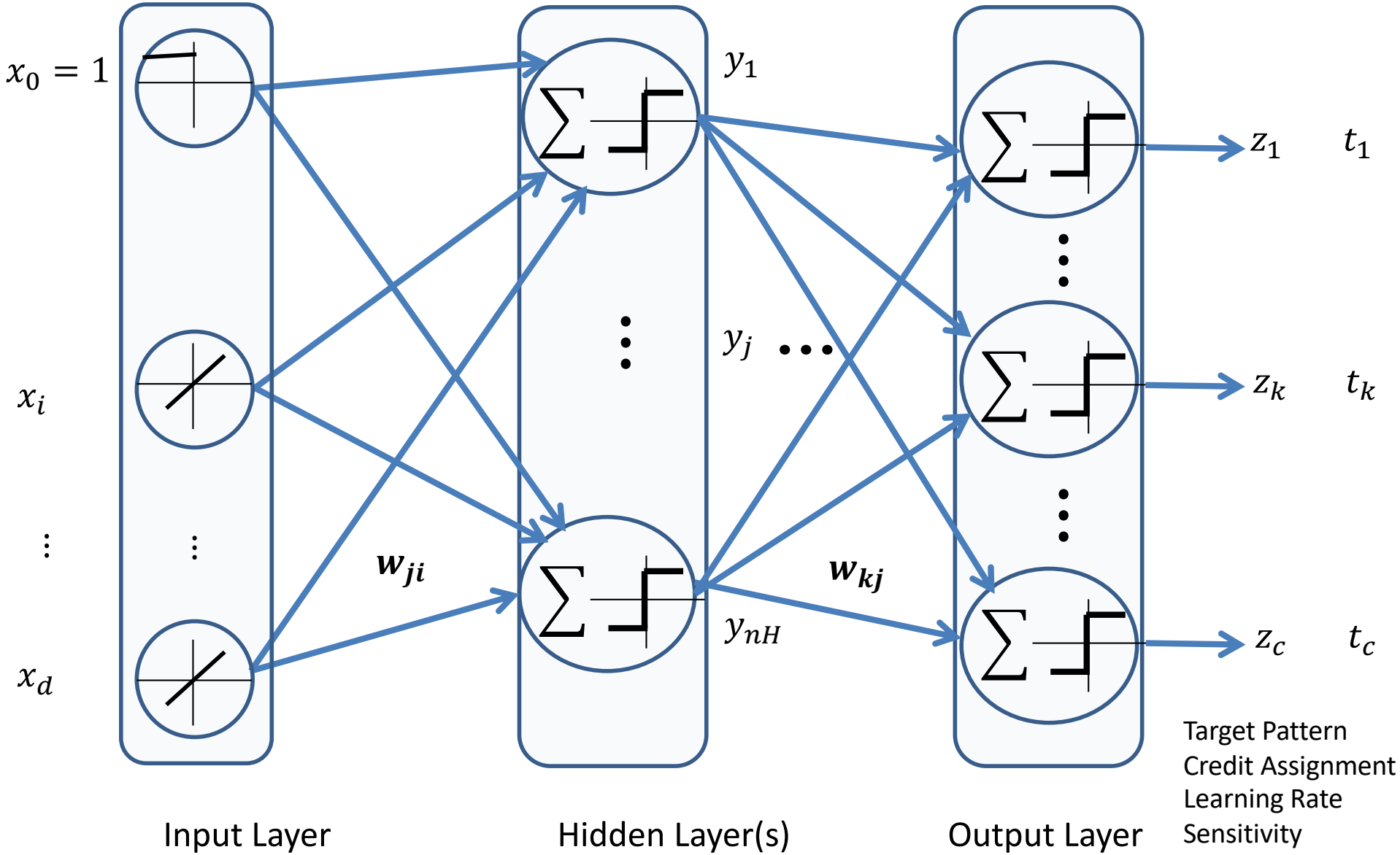
Modelling the Non-linearity with MLP

x	$net_{j=1}$	$net_{j=2}$	y_1	y_2	$net_{k=1}$	z
+2	+3.0	+1.0	+1	+1	-0.5	-1
+1.2	+2.2	+0.2	+1	+1	-0.5	-1
+0.5	+1.5	-0.5	+1	-1	+1.5	+1
0	+1.0	-1.0	+1	-1	+1.5	+1
-0.5	+0.5	-1.5	+1	-1	+1.5	+1
-1.2	-0.2	-2.2	-1	-1	-0.5	-1
-2	-1.0	-3.0	-1	-1	-0.5	-1

Modelling the Non-linearity with MLP



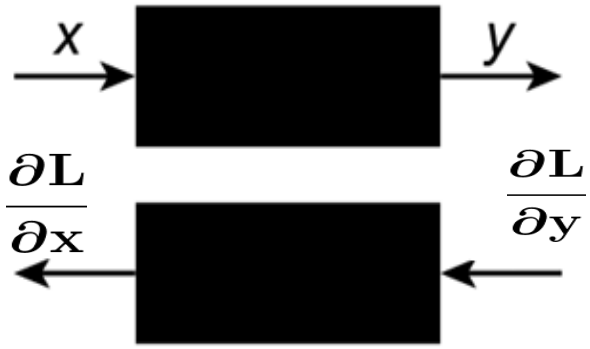
Backpropagation in NN



Target Pattern
Credit Assignment
Learning Rate
Sensitivity

Backpropagation in Neural Networks

Chain Rule of Gradients



Given y(x) and $\frac{\partial \mathbf{L}}{\partial \mathbf{y}}$

What is $\frac{\partial \mathbf{L}}{\partial \mathbf{x}}$ *?*

$$\frac{\partial \mathbf{L}}{\partial \mathbf{x}} =$$

For each layer/parameters, we only need to find $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$

Backpropagation in Neural Networks

- Training Error/ Loss

$$J(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{c}(t_k - z_k)^2 = \frac{1}{2}\|\mathbf{t} - \mathbf{z}\|^2$$

- Gradient $\Delta\mathbf{w} = -\eta\,\frac{\partial J}{\partial \mathbf{w}}$, $\Delta w_{pq} = -\eta\,\frac{\partial J}{\partial w_{pq}}$
- Iterative Weight update: $\boxed{\mathbf{w}(m+1) = \mathbf{w}(m) + \Delta\mathbf{w}}$
- $\Delta w_{kj} = -\eta\,\frac{\partial J}{\partial w_{kj}} = ?$

Backpropagation in Neural Networks

- $\Delta w_{kj} = -\eta \frac{\partial J}{\partial w_{kj}} = -\eta \left(\frac{\partial J}{\partial net_k} * \frac{\partial net_k}{\partial w_{kj}} \right)$
- $\Delta w_{kj} = -\eta \left(-\frac{\partial J}{\partial net_k} * -\frac{\partial net_k}{\partial w_{kj}} \right)$
- $\Delta w_{kj} = -\eta \left(\left(-\frac{\partial J}{\partial z_k} * \frac{\partial z_k}{\partial net_k} \right) * -\frac{\partial net_k}{\partial w_{kj}} \right)$
- $\Delta w_{kj} = -\eta \left(\underline{(t_k - z_k) * f'(net_k)} * -y_j \right)$
- $\Delta w_{kj} = -\eta \left(\underline{\delta_k} * -y_j \right) = \eta \delta_k y_j$

Recall $J(\mathbf{w}) = \frac{1}{2} \|\mathbf{t} - \mathbf{z}\|^2$

Recall $z_k = f(net_k)$

Recall $net_k = \sum_{j=0}^{nH} y_j w_{kj}$

$$\delta_k = -\frac{\partial J}{\partial net_k} = (t_k - z_k) * f'(net_k)$$

Backpropagation in Neural Networks

- $J(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{c}(t_k - z_k)^2 = \frac{1}{2}\|\mathbf{t} - \mathbf{z}\|^2$
- $\Delta\mathbf{w} = -\eta\,\frac{\partial J}{\partial \mathbf{w}}$, $\Delta w_{pq} = -\eta\,\frac{\partial J}{\partial w_{pq}}$
- $\mathbf{w}(m+1) = \mathbf{w}(m) + \Delta\mathbf{w}$
- $\boxed{\Delta w_{kj} = \eta\delta_k y_j = \eta(t_k - z_k)f'(net_k)y_j}$
- $\Delta w_{ji} = -\eta\,\frac{\partial J}{\partial w_{ji}} = ?$

Backpropagation in Neural Networks

Recall $net_j = \sum_{i=0}^{d+1} x_i w_{ji}$

Recall $y_j = f(net_j)$

Recall $net_k = \sum_{j=0}^{nH} y_j w_{kj}$

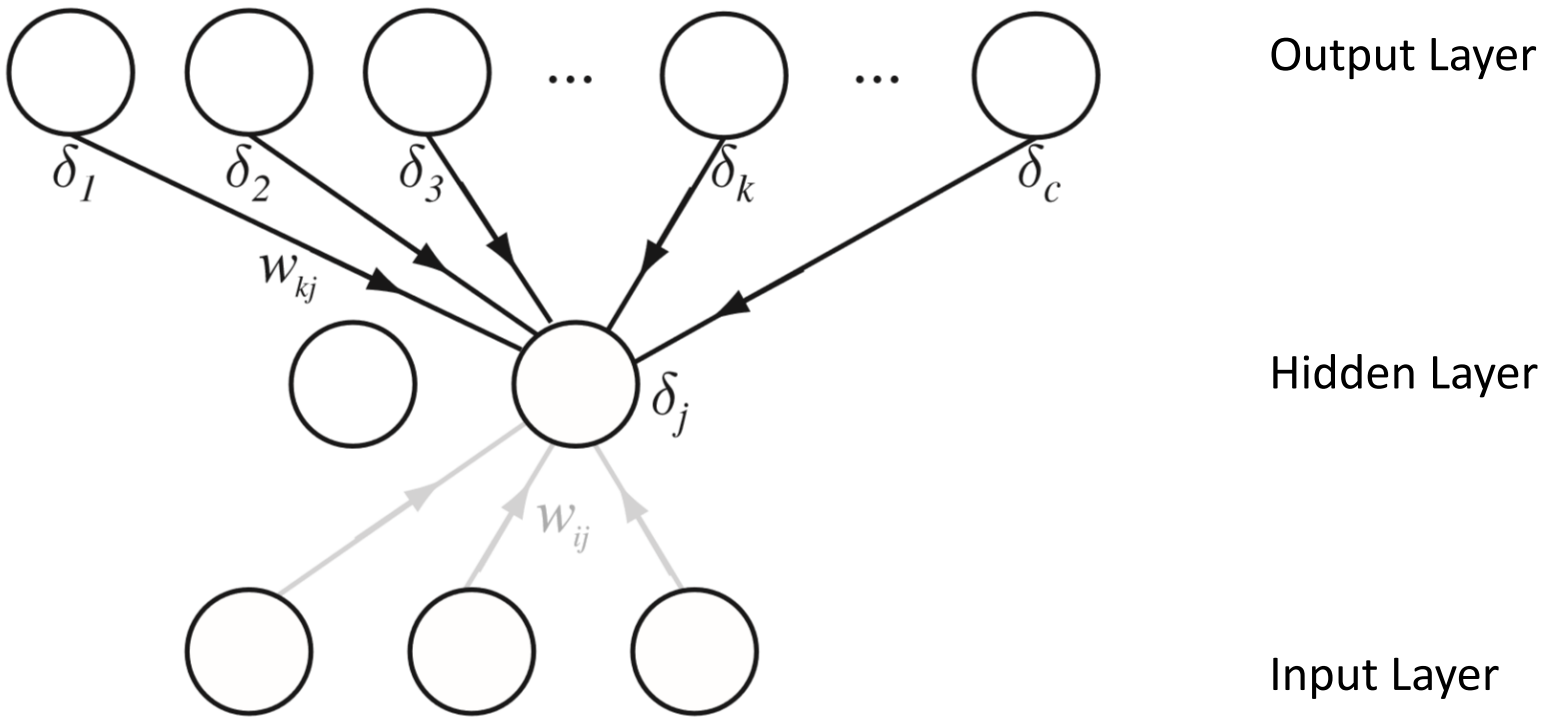
- $\Delta w_{ji} = -\eta \frac{\partial J}{\partial w_{ji}} = -\eta \left(\frac{\partial J}{\partial net_j} * \frac{\partial net_j}{\partial w_{ji}} \right)$
- $\Delta w_{ji} = -\eta \left(\left(\frac{\partial J}{\partial y_j} * \frac{\partial y_j}{\partial net_j} \right) * \frac{\partial net_j}{\partial w_{ji}} \right)$
- $\Delta w_{ji} = -\eta \left(\left(\left(\frac{\partial J}{\partial z_k} * \frac{\partial z_k}{\partial net_k} * \frac{\partial net_k}{\partial y_j} \right) * \frac{\partial y_j}{\partial net_j} \right) * \frac{\partial net_j}{\partial w_{ji}} \right)$
- $\Delta w_{ji} = -\eta \left(\underline{\left(\left(-\sum_{k=1}^{c} (t_k - z_k) * f'(net_k) * w_{kj} \right) * f'(net_j) \right)} * x_i \right)$
- $\Delta w_{ji} = \underline{\eta \delta_j} x_i = \eta \left(\underline{\left[\sum_{k=1}^{c} w_{kj}\, \delta_k \right] f'(net_j)} \right) x_i$

Where, $\delta_k = (t_k - z_k) * f'(net_k)$

Backpropagation in Neural Networks

- Sensitivity of Backpropagation: *How overall error changes with the unit's activation*

$$\delta_k = (t_k - z_k) * f'(net_k), \qquad \delta_j = \left[\sum_{k=1}^{c} w_{kj}\, \delta_k\right] * f'\left(net_j\right)$$



Practical Aspects of Backpropagation

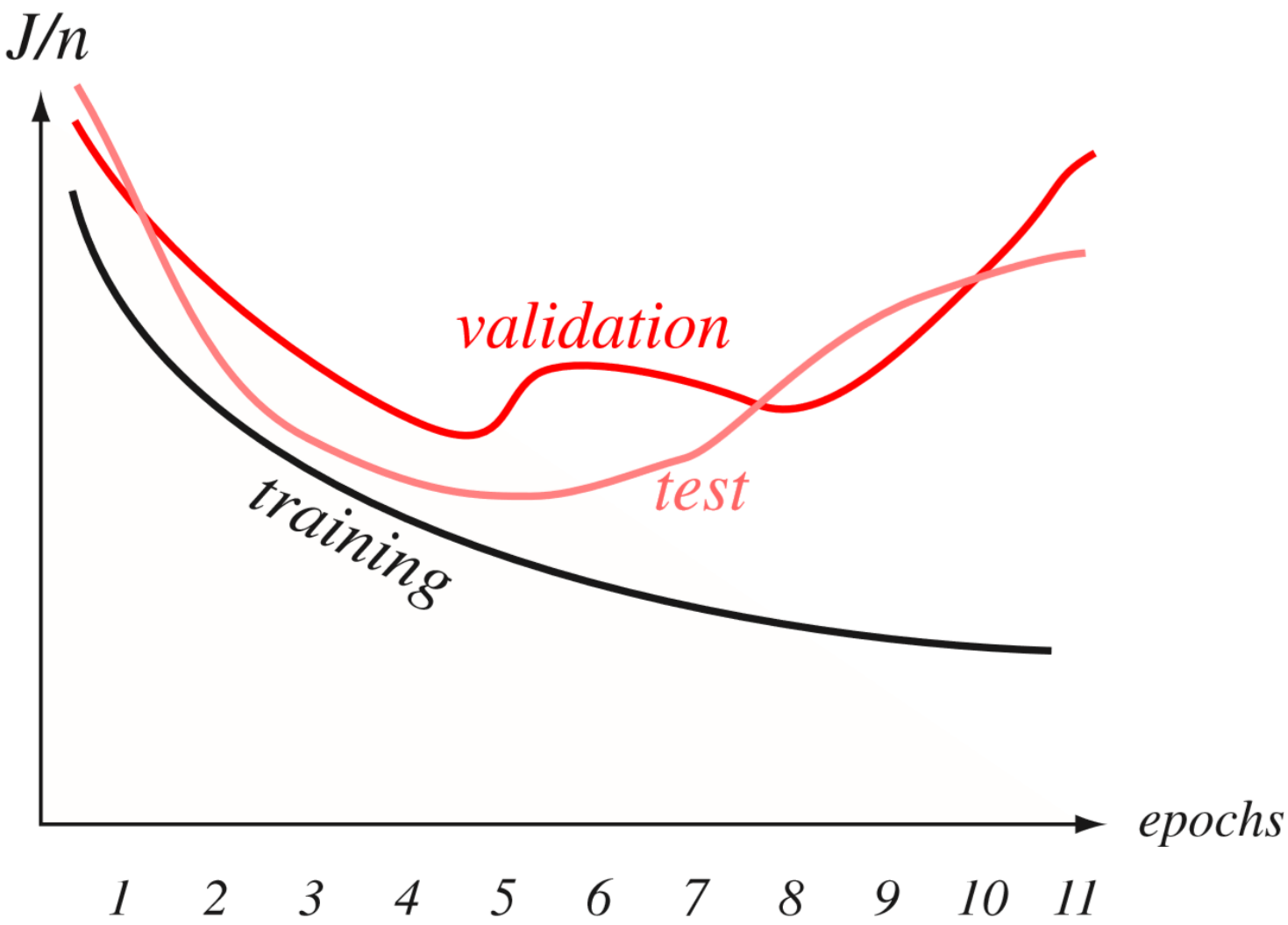
- Training Protocols
  - Stochastic Training – Training samples are chose at random and weight updated for every sample.
  - Batch Training – All training samples in a batch are used at once to compute the loss and update weights.
  - Online Training – Each training sample is used only once for learning the weights.
  - Epoch – The number of presentations of the full training set.

Backpropagation in NN

- Stochastic Backpropagation

1. Initialize $nH, \mathbf{w}, \theta$ (threshold), $\eta, m = 0$
2. do m= $m + 1$
3. randomly choose $\mathbf{x}^m$
4. $w_{ji} = w_{ji} + \eta \delta_j x_i$
5. $w_{kj} = w_{kj} + \eta \delta_k y_j$
6. until $|\nabla J(\mathbf{w})| < \theta$
7. return $\mathbf{w}$

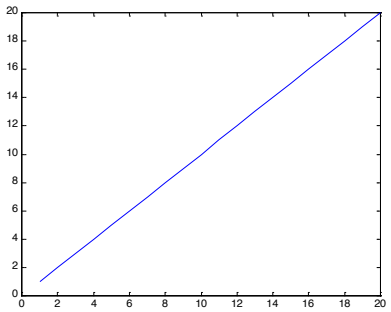
Learning Curves



Practical Aspects of Backpropagation

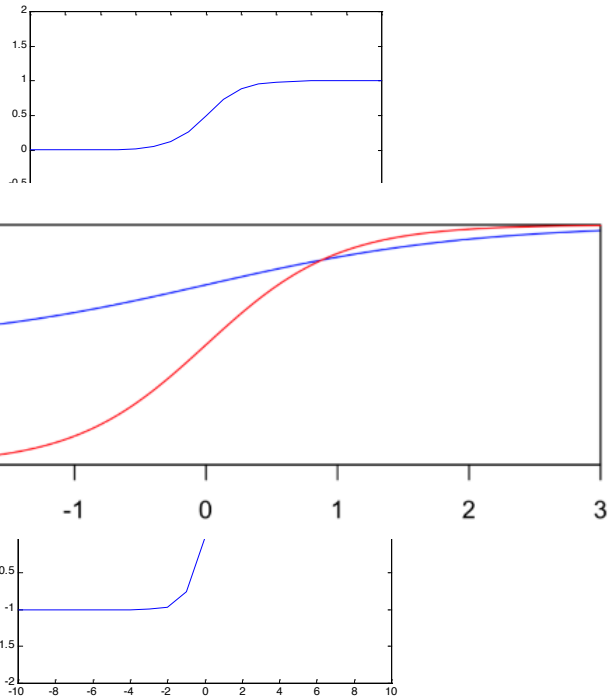
- Activation Function
  - $f(\cdot)$ should be ***Non-linear***
  - $f(\cdot)$ should ***Saturate***
  - $f(\cdot)$ should be ***Continuous & Smooth***
  - $f'(\cdot)$ should be ***Defined***
  - $f(\cdot)$ can have ***Monotonicity***
  - $f(\cdot)$ can be ***linear for small values of*** $\boldsymbol{net}$

Practical Aspects of Backpropagation



Linear

$$y = x$$



Logistic (Sigmoid) O/P : 0 to 1

$$y = \frac{1}{1 + \exp(-x)} = \frac{\exp(x)}{1 + \exp(x)}$$

Hyperbolic Tangent O/P : -1 to +1

$$y = a\left[\frac{\exp(bx) - \exp(-bx)}{\exp(bx) + \exp(-bx)}\right]$$

Practical Aspects of Backpropagation

- Sigmoid as activation function
  - Smooth, Differentiable, Nonlinear and Saturating
  - Admit to linear model for small network weights
  - Derivative of Sigmoid function can be represented in terms of function itself.

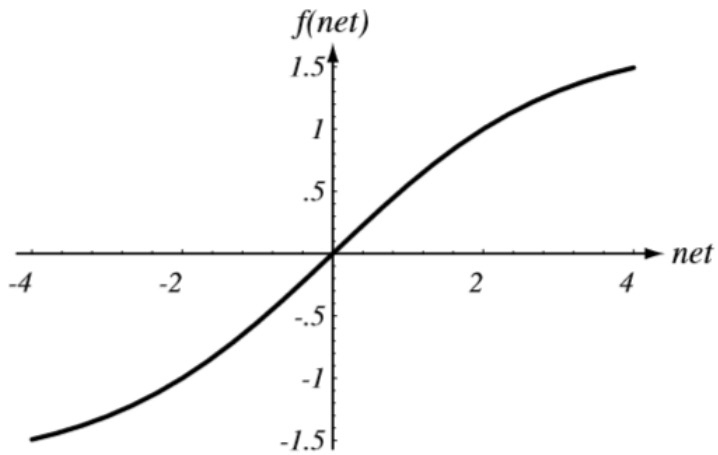
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

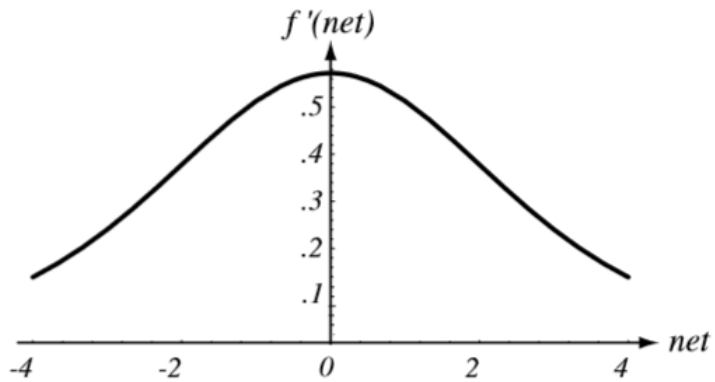
$$\sigma'(x) = -\left\{\frac{1}{(1 + e^{-x})}\right\}^2 e^{-x}\,(-1)$$

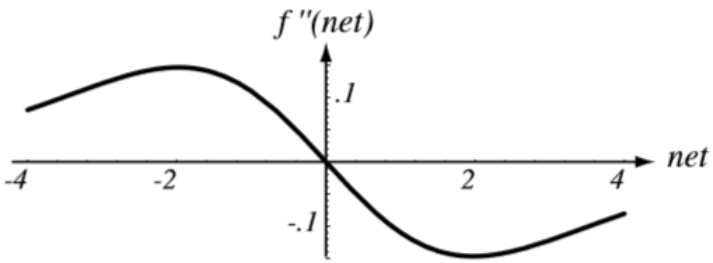
$$\sigma'(x) = \frac{1}{(1 + e^{-x})} \times \frac{e^{-x}}{(1 + e^{-x})}$$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Practical Aspects of Backpropagation







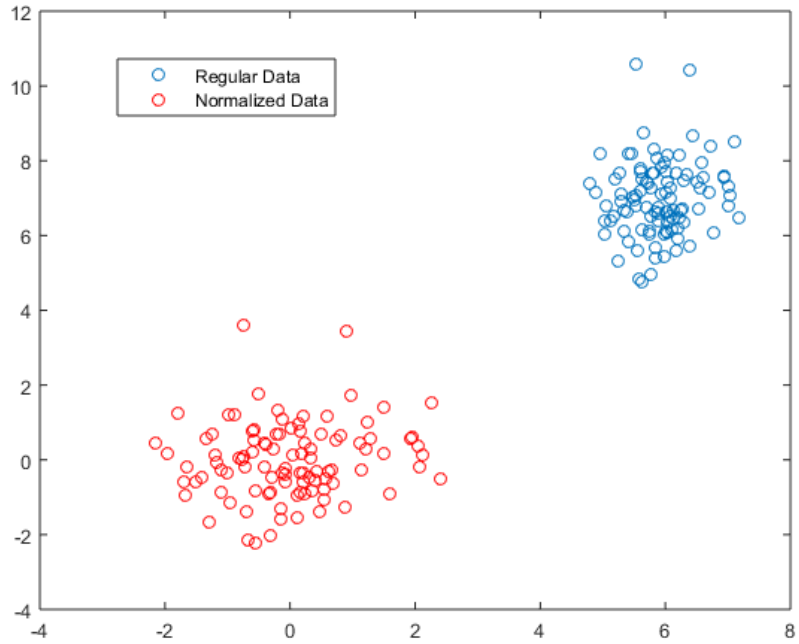
$$f(net) = a \tanh(b\, net)$$

$a = 1.716, b = 2/3$
yields $-1 < net < +1$
and $f''(net)$ has extrema near ± 2

Activation Function centered on zero and asymmetric leads to faster convergence

Practical Aspects of Backpropagation

- **Scaling of Input**
  - $\boldsymbol{X} = [\boldsymbol{x}_1, \cdots, \boldsymbol{x}_m]^T{}_{m \times d}$
  - $\widetilde{\boldsymbol{X}} = \boldsymbol{X} - mean(\boldsymbol{X})$ (Centering of Data)
  - $\boldsymbol{X}_{Norm} = \widetilde{\boldsymbol{X}}/\boldsymbol{\sigma}$ (Column-wise division by Standard Deviation of each dimension)



Practical Aspects of Backpropagation

- **Scaling of Input (*Standardize*)**
  - $\boldsymbol{X} = [\boldsymbol{x}_1, \cdots, \boldsymbol{x}_m]^T{}_{m \times d}$
  - $\widetilde{\boldsymbol{X}} = \boldsymbol{X} - mean(\boldsymbol{X})$ (Centering of Data)
  - $\boldsymbol{X}_{Norm} = \widetilde{\boldsymbol{X}}/\boldsymbol{\sigma}$ (Column-wise division by Standard Deviation of each dimension)
- **Target Values**
  - Use $+1$ and -1 or any real value in this range as output
  - Related to saturation value of the activation function
  - Probabilistic Interpretation is not valid for category labels.
- **Training with Noise**
  - Add random noise to original training samples for generating more training samples

Practical Aspects of Backpropagation

- **Manufacturing Data (Augmentation)**
  - Add translation and rotation transforms to original training data to generate more rich training data samples
- **Number of Hidden Units**
  - Too few leads to high test error due to lack of expressibility
  - Too many leads to overfitting to training data
  - Choose such that total number of weights = $n/10$.
- **Weight Initialization**
  - Do not initialize with zero weights
  - Use both random positive & negative weights as data is standardized
  - Consider the saturation value of net activation function while choosing range for initial weights.
  - $-1/\sqrt{d} < w_{ji} < +1/\sqrt{d}$ and $-1/\sqrt{nH} < w_{kj} < +1/\sqrt{nH}$

Thank You